



Final Report

Domain-Specific Fitness NLP Assistant Using Retrieval-Augmented Generation

Domen Pahole, Luka Rizman, Nina Trivić

Abstract

We present a domain-specific fitness assistant built around Retrieval-Augmented Generation (RAG). The system combines a sentence-transformer retriever, a Qdrant vector database, and the Llama 3.2 1B model served through Ollama. Our main goal is not to maximize raw language fluency, but to improve grounding to a fixed exercise dataset and reduce unsupported recommendations. On an 11-query retrieval benchmark, the system reaches Hit@1 = 0.636, Hit@3 = 0.909, Hit@5 = 0.909, Precision@5 = 0.582, and MRR = 0.742. Qualitative comparison against the same 1B model without retrieval shows that RAG produces more dataset-aligned answers when retrieval succeeds, but also inherits retrieval failures. The main take-away is that, for this project, retrieval quality is the dominant bottleneck and the most promising direction for future improvement.

Keywords

Natural Language Processing, Retrieval-Augmented Generation, Fitness Assistant, Semantic Retrieval, Large Language Models

Advisors: Aleš Žagar

1 Introduction

General-purpose LLMs are often fluent, but they are not reliable domain experts when answers depend on a specific local knowledge base. Our project addresses this problem in the fitness domain. We build a conversational assistant that answers questions about exercises, equipment, muscle groups, and training suggestions using a structured exercise dataset rather than relying only on the model's parametric knowledge.

The project follows the course topic of domain-specific AI assistants. We define a narrow use case, prepare a domain knowledge base, implement retrieval and generation components, and evaluate both retrieval quality and end-to-end answer behavior. The final system is designed to be easy to rerun locally or on HPC, with all dependencies and scripts available in the repository.

2 Data and System

Dataset. We use the MegaGym exercise dataset, which contains 2,918 exercise entries. Each entry includes a title, description, exercise type, target body part, equipment, difficulty level, and optional rating fields. The dataset is public and linked in the repository instructions, so it is not duplicated in version control.

Knowledge representation. Each exercise is converted into a textual record containing the title, description, type, body part, equipment, difficulty level, and rating metadata. These records are embedded and stored in Qdrant.

Retriever. We use the Sentence Transformers model all-MiniLM-L6-v2. Retrieval uses cosine similarity in Qdrant with default depth $\text{top-}k = 5$.

Generator. For answer generation we use Llama 3.2 1B through Ollama. This choice follows the course constraint to use a small model that can also run on HPC. The same model is used in both baseline and RAG settings, so the comparison isolates retrieval effects rather than model size.

Prompting. In the RAG setting, the prompt instructs the model to use only retrieved exercise entries, prefer matches by body part, equipment, and difficulty, remove duplicates, and explicitly state when the dataset is insufficient. The baseline prompt receives only the user question.

Implementation details. The repository contains a Streamlit interface, indexing script, retrieval evaluation script, baseline-vs-RAG comparison script, and a unified evaluation runner. The system can run locally with Docker-based Qdrant or on Arnes HPC with local Qdrant storage and Ollama on a GPU node.

Table 1. Algorithm settings used for all reported results. The table is self-contained: it specifies the model choices, retrieval setup, and operational hyperparameters needed to rerun the same pipeline.

Component	Final setting
Retriever encoder	Sentence Transformers all-MiniLM-L6-v2 (384-dimensional embeddings)
Vector database and search	Qdrant collection megagym_exercises, cosine similarity
Retrieval depth	Top- $k = 5$ for app responses and evaluation scripts
Main improvement in prompting	Use only retrieved entries, prioritize body part/equipment/difficulty matches, remove near-duplicates, and explicitly state when context is insufficient
Generator	llama3.2:1b via Ollama (same model in baseline and RAG)
Indexing/runtime hyperparameters	Embedding batch size 64, Qdrant upsert chunk size 512, Ollama request timeout 180s

3 Evaluation Setup

We evaluate the system in two complementary ways.

Retrieval evaluation. We created a small benchmark of 11 natural-language fitness queries with manually specified expectations. Each query defines relevant properties such as body part, equipment, difficulty level, and keywords. Retrieved items are marked relevant if they satisfy these constraints. We report Hit@1, Hit@3, Hit@5, Precision@5, and Mean Reciprocal Rank (MRR).

End-to-end comparison. We compare baseline generation and RAG generation on five representative questions. For each question, we manually select the better answer based on constraint alignment (body part, equipment, difficulty) and dataset grounding. This gives a compact per-query winner table while keeping the same model fixed in both settings.

4 Results

Table 2 summarizes the main quantitative results of the retrieval component.

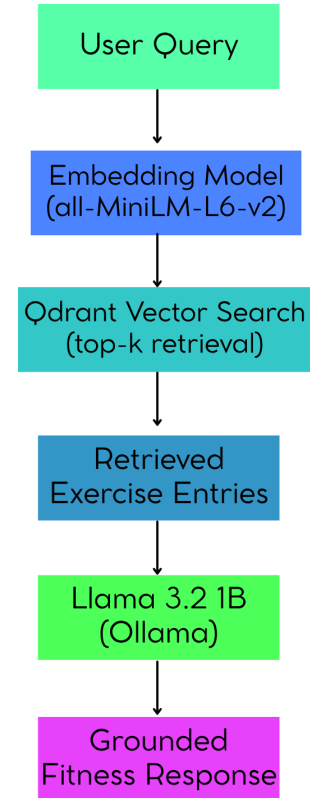


Figure 1. RAG pipeline used in the fitness assistant: user query, retrieval from Qdrant, and grounded generation with Llama 3.2 1B.

The results show that retrieval is already strong enough to support a useful RAG pipeline. In 10 out of 11 queries, at least one relevant exercise appears in the top five retrieved items. However, the lower Hit@1 score shows that the first retrieved result is still often suboptimal. This matters because the generator is sensitive to noisy or weak context.

Table 3 summarizes the end-to-end comparison in one consolidated format so the reader can compare systems query-by-query.

Related prior work in fitness recommendation [1] is contextually relevant but not directly comparable numerically here because it targets a different setup and does not report MegaGym retrieval metrics such as Hit@ k or MRR.

5 Analysis of Successes and Failures

The clearest success case is the query “Can you recommend some triceps exercises that use a cable machine?” In the baseline setting, the model gives plausible but generic cable-triceps suggestions. In the RAG setting, the answer is more tightly aligned with the dataset and mentions retrieved entries such as *Cable Straight-Bar Push-Down* and *Triceps Overhead Extension with Rope*. This is exactly the intended benefit of RAG: the same small model becomes more domain-grounded when given relevant context.

One representative failure case is “Which bodyweight leg

Table 2. Retrieval evaluation on 11 manually specified fitness queries. Higher is better for all metrics. The table is self-contained: Hit@ k measures whether at least one relevant exercise appears in the top k retrieved items, Precision@5 measures the fraction of relevant items among the top five, and MRR measures how early the first relevant result appears.

Metric	Value
Queries	11
Hit@1	0.636
Hit@3	0.909
Hit@5	0.909
Precision@5	0.582
MRR	0.742

exercises would you suggest for a beginner?" Retrieval fails here because the returned items are only partially aligned with the intended constraints. Some retrieved exercises are plyometric, some have missing equipment information, and one result even targets abdominals rather than legs. As a consequence, the RAG answer inherits the retrieval error and becomes less useful than the baseline. This example is important because it shows that RAG does not automatically improve every answer; it improves answers only when retrieval is good enough.

We also observe partial failures caused by noisy metadata and duplicates. For example, some retrieved titles are near-duplicates, and some exercise descriptions are repeated or inconsistently labeled. This reduces Precision@5 and makes the prompt context less clean than it should be.

6 Discussion

The main conclusion is that retrieval quality dominates final answer quality in this project. Our main improvement over baseline is the grounding-constrained RAG prompt: the model is forced to use retrieved entries, prioritize body part/equipment/difficulty matches, remove duplicates, and admit insufficient evidence. This improves specificity when retrieval is good and makes failure cases easier to diagnose when retrieval is noisy.

This finding is practically useful for future work. The most promising improvements are not larger models, but better retrieval and cleaner data. In particular, the next steps should include:

- metadata-aware filtering by body part, equipment, and difficulty,
- dataset cleaning and deduplication,
- expansion of the evaluation benchmark beyond 11 retrieval queries,
- more systematic human evaluation of final answers.

Table 3. Consolidated baseline-vs-RAG comparison on five representative questions using the same Llama 3.2 1B model. “Better” is decided by manual judgment of alignment with query constraints and grounding in retrieved MegaGym entries. The last row reports total wins; the best total is bolded.

Query	Better	Evidence
Triceps with cable	RAG	Uses cable-triceps exercises directly matching retrieved dataset entries, while baseline remains generic.
Beginner bodyweight legs	Baseline	Retrieved set is partially mismatched (plyometrics and off-target items), so RAG inherits retrieval errors.
Back with pull-up bar	Baseline	RAG includes off-target suggestions (e.g., squat), while baseline stays closer to the requested equipment.
Biceps with barbell	RAG	More grounded barbell naming from retrieved entries; baseline includes broader generic advice.
Abs with exercise ball	RAG	Better dataset-grounded naming of exercise-ball abdominal variants.
Total wins (5 queries)	Baseline 2 / RAG 3	Best overall end-to-end score in this project is RAG.

Our work is also consistent with the general motivation of RAG systems [2]: retrieval can inject task-specific knowledge into a fixed generator. In the fitness domain, related work has also explored exercise recommendation systems that combine structured information and NLP components [1]. Our contribution is smaller in scale, but it demonstrates the same core principle in a reproducible course project setting.

7 Reproducibility and Final Take-Away

The repository is structured so that another student can rerun the project with minimal setup. Dependencies are pinned in `requirements.txt`; local setup and HPC setup are documented in `LOCAL_SETUP.md` and `peer-review.md`. The MegaGym dataset is linked (not versioned), indexing and evaluation scripts are included, and results are written to `report/results/`. This satisfies the requirement that reviewers can understand and rerun the full pipeline.

Take-away message. With the same small 1B language model, RAG improves domain grounding by forcing answers to use retrieved MegaGym exercise entries. The main limitation is not the generator itself, but retrieval quality: when retrieval is strong, answers become more specific and trustworthy; when retrieval is weak, the whole system fails with it.

References

- [1] Yuer Yang, Yifeng Lin, Zeguang Chen, Yongjia Lei, Xingxing Liu, Yan Zhang, Ying Sun, and Xiangjie Wang. Snpers: A physical exercise recommendation system integrating statistical principles and natural language processing. *Electronics*, 12(1):61, 2023.
- [2] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.